WEB TECHNOLOGY

Important Questions — Exam Answer Handbook

16 exam-oriented answers • definitions • examples • setup steps • complete programs

SECTION	QUESTIONS	ANSWER DEPTH
Very short	1–5	Definition + key facts
5 marks	6–12	Explanation + example / steps
10 marks	13–16	Program + explanation + execution

Exam method: For 5 marks, write a definition, 4–5 points and an example. For 10 marks, include the algorithm or architecture, properly indented code, working, output where relevant, and a short conclusion.

Code assumptions: Node.js with npm, Express, and the official MongoDB Node.js driver. Keep connection strings in environment variables; never commit secrets.

Contents

PART A — Very Short: 1 Node.js environment; 2 Stateful vs stateless beans; 3 MongoDB features; 4 DOM; 5 networking and URL terms.

PART B — 5 marks: 6 npm; 7 REPL; 8 JavaBeans; 9 AJAX; 10 full project setup; 11 JavaScript properties/functions/scripts; 12 dynamic content and AJAX.

PART C — 10 marks: 13 insert one/many MongoDB records; 14 CRUD web app; 15 array sorting; 16 registration validation.

Terminology note: ‘stateful/stateless beans’ is interpreted as Enterprise Java session beans (EJB). JavaBeans are covered separately in Question 8.

PART A — VERY SHORT ANSWERS

1. What is the Node.js environment?

Node.js is an open-source, cross-platform JavaScript runtime that executes JavaScript outside a browser. It uses Chrome's V8 engine and is used for servers, REST APIs, command-line tools and real-time applications.

- Event-driven: responds to events such as HTTP requests and file operations.
- Non-blocking I/O: asynchronous I/O lets the process handle other work while waiting.
- Package ecosystem: npm installs reusable packages and manages dependencies.

Example: run a file with `node app.js`.

2. Differentiate between stateful and stateless beans

A session bean contains business logic in an Enterprise Java application. The main distinction is whether it keeps conversational state for a particular client.

Basis	Stateful session bean	Stateless session bean
State	Retains client-specific conversational state across calls.	Does not retain client-specific conversational state between calls.
Instances	Typically associated with a client session.	Instances may be pooled and reused.
Resources	Can use more memory per active conversation.	Often scales well for independent requests.
Example	Shopping cart or multi-step checkout.	Currency conversion or catalogue lookup.

Do not confuse EJB session beans with JavaBeans, which are reusable Java classes following conventions.

3. Key features of MongoDB

MongoDB is a document-oriented NoSQL database. It stores data as BSON documents inside collections.

- Flexible document schema; records can have different fields.
- Rich queries, projections, sorting and aggregation pipeline.
- Indexes improve query performance when chosen appropriately.
- Replica sets provide redundancy and failover.
- Sharding can distribute data horizontally.
- Supports atomic single-document operations and multi-document transactions.

Example document: `{ name: "Asha", age: 20, skills: ["JS", "Node"] }`.

4. What is the DOM (Document Object Model)?

The DOM is a programming interface that represents an HTML/XML document as a hierarchical tree of objects. JavaScript can read, modify, add or remove elements and respond to user actions.

Example

```
document.getElementById("message").textContent = "Welcome!";
```

This finds the element with id `message` and changes its text without requiring a full page reload.

5. Explain TCP server socket, Internet connection, IP address, URL, URLConnection, datagram sockets and client-server computing

Term	Meaning
TCP server socket	A server endpoint that listens for and accepts reliable, ordered TCP connections.
Internet connection	A network connection that enables communication with other networks using Internet protocols.
IP address	A network-layer address identifying an interface, e.g. IPv4 192.0.2.10 or an IPv6 address.
URL	Uniform Resource Locator; identifies how to locate a resource, e.g. https://example.com/docs.
URLConnection	Java API abstraction for communicating with a URL resource; created with URL.openConnection().
Datagram socket	Connectionless socket that sends/receives packets, commonly over UDP; delivery and order are not guaranteed.
Client-server computing	Clients request services/resources; servers process requests and return responses.

PART B — 5-MARK ANSWERS

6. Explain npm and how to install and initialise npm

npm (Node Package Manager) is distributed with Node.js and manages packages, dependency versions and project scripts. It records project metadata in `package.json`.

- Install the Node.js LTS release; npm is included.
- Verify installation using `node -v` and `npm -v`.
- Create and enter a folder: `mkdir my-app`, then `cd my-app`.
- Run `npm init -y` for default settings, or `npm init` to answer prompts.
- Install a dependency with `npm install express`; npm creates/updates package-lock.json.
- Run scripts from package.json, e.g. `npm start`.

Command sequence

```
mkdir my-app
cd my-app
npm init -y
npm install express
node -v
npm -v
```

The lockfile records resolved dependency versions, helping make installations reproducible.

7. What is a REPL terminal?

REPL means Read–Eval–Print Loop. It is an interactive environment that reads a command, evaluates it, prints the result and waits for the next command. Node.js provides a built-in JavaScript REPL.

Stage	Action
Read	Accept a command or expression.
Evaluate	Execute the expression.
Print	Display result or error.
Loop	Wait for the next expression.

Example

```
$ node
> 2 + 3
5
> const name = "Asha"
undefined
> name.toUpperCase()
'ASHA'
```

Uses: quick experiments, learning APIs, debugging small expressions. Exit with `.exit` or Ctrl+D.

8. What are JavaBeans? Explain features with an example

A JavaBean is a reusable Java class that encapsulates data using conventions. A conventional bean has private properties, public getters/setters and a public no-argument constructor. It may implement Serializable when persistence/transfer is required.

- Encapsulation: private fields accessed through methods.
- Reusability: can be used by different application components and tools.
- Introspection: tools can identify properties through naming conventions.
- Customisation: properties can be set through setters.
- Persistence: serialization may be used when appropriate, but is not universally mandatory.

StudentBean.java

```
import java.io.Serializable;

public class StudentBean implements Serializable {
    private String name;
    private int age;

    public StudentBean() {}
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public int getAge() { return age; }
    public void setAge(int age) { this.age = age; }
}
```

Using the bean

```
StudentBean s = new StudentBean();
s.setName("Asha");
s.setAge(20);
System.out.println(s.getName() + " " + s.getAge());
```

EJB session beans are different enterprise components; JavaBeans are convention-based reusable classes.

9. What is AJAX? Explain its advantages with an example

AJAX means Asynchronous JavaScript and XML. It is a technique for exchanging data with a server asynchronously and updating part of a page without reloading the entire document. Modern applications commonly use JSON instead of XML.

- Updates only the relevant page section.
- Improves perceived responsiveness.
- Avoids transferring unchanged page content.
- Allows background requests while users continue interacting.
- Can exchange JSON, XML, text or other supported formats.

Modern AJAX example using Fetch

```
document.getElementById("load").addEventListener("click", async () => {
  const response = await fetch("/api/message");
  if (!response.ok) throw new Error("Request failed");
  const data = await response.json();
  document.getElementById("result").textContent = data.message;
});
```

The browser requests data, parses the JSON response and updates the DOM. AJAX is a technique, not a single technology.

10. Set up an application using JavaScript + Node.js + Express + MongoDB

JavaScript supplies application logic, Node.js runs it, Express handles HTTP routes/middleware, and MongoDB stores documents.

- Install Node.js LTS and npm. Install MongoDB locally or create a MongoDB Atlas deployment.
- Create a folder and run `npm init -y`.
- Install packages: `npm install express mongodb dotenv`.
- Create `server.js` and `.env`. Put the database URI in .env; never commit secrets.
- For local MongoDB use `MONGODB_URI=mongodb://127.0.0.1:27017`. For Atlas, use its supplied URI.
- Connect with MongoClient, define Express routes, then listen on a port.
- Run `node server.js` and test `http://localhost:3000/health` using a browser or Postman.
- Add input validation, error handling, logging, authentication and graceful shutdown for real deployments.

server.js — minimal server

```
require("dotenv").config();
const express = require("express");
const { MongoClient } = require("mongodb");
const app = express();
app.use(express.json());

const client = new MongoClient(process.env.MONGODB_URI);

async function start() {
  await client.connect();
  const db = client.db("webtech");
  app.get("/health", (req, res) => res.json({ status: "ok" }));
  app.get("/students", async (req, res, next) => {
    try {
      const rows = await db.collection("students")
        .find({}).limit(100).toArray();
      res.json(rows);
    } catch (err) { next(err); }
  });
  app.listen(3000, () => console.log("http://localhost:3000"));
}
start().catch(err => {
  console.error("Startup failed:", err);
  process.exit(1);
});
```

.env and commands

```
MONGODB_URI=mongodb://127.0.0.1:27017

npm init -y
npm install express mongodb dotenv
node server.js
```

Request flow: browser/Postman → Express route → MongoDB driver → MongoDB → JSON response. The database server must be running and reachable.

11. Properties of JavaScript, predefined functions, inline and external JavaScript

JavaScript is a high-level, dynamically typed, prototype-based language with first-class functions. It is used in browsers and in server runtimes such as Node.js.

- Dynamic typing: a variable can hold values of different types.
- Prototype-based object model; class syntax is built on prototypes.
- First-class functions and closures.
- Asynchronous/event-driven programming with promises and async/await.
- Case-sensitive; follows ECMAScript language rules.
- JavaScript execution is often single-threaded in a given environment, while I/O can proceed asynchronously.

Built-in / method	Purpose	Example
alert()	Browser dialog	alert("Hello")
prompt()	Ask user for input	prompt("Name?")
parseInt()	Parse integer	parseInt("42", 10)
Number()	Convert to number	Number("3.5")
isNaN()	Test NaN after conversion	isNaN("abc")
Math.max()	Largest number	Math.max(2, 9, 4)
Array.sort()	Sort array in place	[3,1,2].sort((a,b)=>a-b)

Note: alert() and prompt() are browser APIs; parseInt(), Number(), isNaN() and Math methods are JavaScript built-ins.

Inline JavaScript

Inline event attribute (brief demonstration)

```
<button onclick="alert('Hello!')">Click</button>
```

Quick for demonstrations, but it mixes behaviour with HTML and is harder to maintain. Prefer event listeners in real applications.

Internal and external JavaScript

Internal script in HTML

```
<script>
  document.getElementById("msg").textContent = "Hello";
</script>
```

External app.js

```
document.getElementById("btn").addEventListener("click", () => {
  alert("Button clicked");
});
```

Load the external file

```
<button id="btn">Click</button>
<script src="app.js" defer></script>
```

External files improve reuse, separation of concerns and caching. The defer attribute runs a classic external script after HTML parsing.

12. What is dynamic content and how does AJAX contribute?

Dynamic content changes in response to user actions, server data, time, preferences or application state rather than remaining fixed HTML.

- Examples: cart totals, live search suggestions, dashboard statistics and newly submitted comments.
- JavaScript sends an asynchronous request using Fetch or XMLHttpRequest.
- The server processes the request and returns data, often JSON.
- JavaScript reads the response and updates relevant DOM elements without reloading the whole page.

Example

```
async function loadCount() {
  const res = await fetch("/api/count");
  if (!res.ok) throw new Error("Request failed");
  const data = await res.json();
  document.getElementById("count").textContent = data.count;
}
```

AJAX can improve responsiveness but still requires loading states, error handling, security and accessible status messages.

PART C — 10-MARK ANSWERS

13. JavaScript program to insert one record and many records into MongoDB

Use the official MongoDB Node.js driver. `insertOne()` inserts one document; `insertMany()` inserts an array of documents.

- Prerequisites: MongoDB server or Atlas URI, Node.js/npm.
- Install packages: `npm init -y` and `npm install mongodb dotenv`.
- Put the connection string in .env as MONGODB_URI; do not commit it.

insert-records.js

```
require("dotenv").config();
const { MongoClient } = require("mongodb");

async function main() {
  const client = new MongoClient(process.env.MONGODB_URI);
  try {
    await client.connect();
    const students = client.db("collegeDB").collection("students");

    const one = {
      rollNo: 101, name: "Asha", branch: "CSE", semester: 5
    };
    const singleResult = await students.insertOne(one);
    console.log("Inserted one:", singleResult.insertedId);

    const many = [
      { rollNo: 102, name: "Ravi", branch: "CSE", semester: 5 },
      { rollNo: 103, name: "Neha", branch: "IT", semester: 5 },
      { rollNo: 104, name: "Kabir", branch: "ECE", semester: 5 }
    ];
    const manyResult = await students.insertMany(many);
    console.log("Inserted count:", manyResult.insertedCount);
    console.log("Inserted IDs:", manyResult.insertedIds);
  } catch (err) {
    console.error("Database operation failed:", err.message);
  } finally {
    await client.close();
  }
}
main().catch(console.error);
```

Execute

```
node insert-records.js
```

Explanation

- MongoClient connects to the database server.
- db() selects the database; collection() accesses the collection.
- insertOne() inserts one document and returns its inserted ID.
- insertMany() inserts multiple documents and reports count/IDs.
- finally closes the client even if an operation fails.

This inserts one plus three documents into collegeDB.students. Repeated execution creates duplicates unless uniqueness or duplicate-check logic is added.

14. Design a web application to perform CRUD operations

CRUD means Create, Read, Update and Delete. This educational application uses Node.js, Express, MongoDB and a browser interface to manage student records.

A. Install and configure

Commands and .env

```
mkdir student-crud
cd student-crud
npm init -y
npm install express mongodb dotenv

# .env
MONGODB_URI=mongodb://127.0.0.1:27017
```

B. Server — server.js

Express API with all four CRUD operations

```
require("dotenv").config();
const express = require("express");
const { MongoClient, ObjectId } = require("mongodb");
const path = require("path");

const app = express();
app.use(express.json());
app.use(express.static(path.join(__dirname, "public")));
const client = new MongoClient(process.env.MONGODB_URI);
let students;
const validId = id => ObjectId.isValid(id);

async function start() {
  await client.connect();
  students = client.db("collegeDB").collection("students");

  // CREATE
  app.post("/api/students", async (req, res, next) => {
    try {
      const { name, branch, semester } = req.body;
      if (typeof name !== "string" || !name.trim())
        return res.status(400).json({ error: "Name required" });
      const doc = {
        name: name.trim(),
        branch: typeof branch === "string" ? branch.trim() : "",
        semester: Number(semester) || 1
      };
      const r = await students.insertOne(doc);
      res.status(201).json({ _id: r.insertedId, ...doc });
    } catch (e) { next(e); }
  });

  // READ
  app.get("/api/students", async (req, res, next) => {
    try {
      res.json(await students.find({}).sort({ name: 1 })
        .limit(200).toArray());
    } catch (e) { next(e); }
  });

  // UPDATE
  app.put("/api/students/:id", async (req, res, next) => {
    try {
      if (!validId(req.params.id))
        return res.status(400).json({ error: "Invalid ID" });
      const { name, branch, semester } = req.body;
      if (typeof name !== "string" || !name.trim())
        return res.status(400).json({ error: "Name required" });
      const r = await students.updateOne(
        { _id: new ObjectId(req.params.id) },
        { $set: {
          name: name.trim(),
          branch: typeof branch === "string" ? branch.trim() : "",
          semester: Number(semester) || 1
        }}
      );
      if (!r.matchedCount) return res.status(404).json({ error: "Not found" });
      res.json({ message: "Student updated" });
    } catch (e) { next(e); }
  });

  // DELETE
  app.delete("/api/students/:id", async (req, res, next) => {
    try {
      if (!validId(req.params.id))
        return res.status(400).json({ error: "Invalid ID" });
      const r = await students.deleteOne({
        _id: new ObjectId(req.params.id)
      });
      if (!r.deletedCount) return res.status(404).json({ error: "Not found" });
      res.json({ message: "Student deleted" });
    } catch (e) { next(e); }
  });

  app.use((err, req, res, next) => {
    console.error(err);
    res.status(500).json({ error: "Internal server error" });
  });
  app.listen(3000, () => console.log("Open http://localhost:3000"));
}
start().catch(err => {
```

```
  console.error("Startup failed:", err);
  process.exit(1);
});
process.on("SIGINT", async () => {
  await client.close();
  process.exit(0);
});
```

C. Browser UI — public/index.html

HTML + JavaScript interface

```
<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Student CRUD</title>
<style>
body {font:16px Arial;max-width:850px;margin:32px auto;padding:0 16px}
form {display:flex;gap:8px;flex-wrap:wrap;margin-bottom:20px}
input,button {padding:9px} li {margin:10px 0}
</style>
</head>
<body>
<h1>Student Records</h1>
<form id="studentForm">
<input id="name" placeholder="Name" required>
<input id="branch" placeholder="Branch">
<input id="semester" type="number" min="1" max="12" value="5">
<button type="submit">Add student</button>
</form>
<p id="status" role="status"></p><ul id="list"></ul>
<script>
const form = document.getElementById("studentForm");
const list = document.getElementById("list");
const statusBox = document.getElementById("status");

async function request(url, options = {}) {
  const res = await fetch(url, {
    headers: {"Content-Type":"application/json"}, ...options
  });
  const data = await res.json();
  if (!res.ok) throw new Error(data.error || "Request failed");
  return data;
}
async function loadStudents() {
  try {
    const rows = await request("/api/students");
    list.replaceChildren();
    rows.forEach(s => {
      const li = document.createElement("li");
      const label = document.createElement("span");
      label.textContent = `${s.name} — ${s.branch} — Sem ${s.semester} `;
      const edit = document.createElement("button");
      edit.textContent = "Edit";
      edit.onclick = async () => {
        const name = prompt("Name", s.name);
        if (name === null || !name.trim()) return;
        const branch = prompt("Branch", s.branch) ?? s.branch;
        const semester = prompt("Semester", s.semester) ?? s.semester;
        try {
          await request("/api/students/" + s._id, {
            method:"PUT", body:JSON.stringify({name,branch,semester})
          });
          await loadStudents();
        } catch(e) { statusBox.textContent = e.message; }
      };
      const del = document.createElement("button");
      del.textContent = "Delete";
      del.onclick = async () => {
        if (!confirm("Delete this record?")) return;
        try {
          await request("/api/students/" + s._id, {method:"DELETE"});
          await loadStudents();
        } catch(e) { statusBox.textContent = e.message; }
      };
      li.append(label, edit, del); list.append(li);
    });
  } catch(e) { statusBox.textContent = e.message; }
}
form.addEventListener("submit", async e => {
  e.preventDefault();
  const payload = {
    name:document.getElementById("name").value,
    branch:document.getElementById("branch").value,
    semester:document.getElementById("semester").value
  };
  try {
    await request("/api/students", {
      method:"POST", body:JSON.stringify(payload)
    });
    form.reset(); statusBox.textContent = "Student added.";
```

```
    await loadStudents();
  } catch(e) { statusBox.textContent = e.message; }
});
loadStudents();
</script>
</body></html>
```

D. Run and CRUD mapping

Run

```
node server.js
# Open http://localhost:3000
```

Action	HTTP route	MongoDB operation
Create	POST /api/students	insertOne()
Read	GET /api/students	find()
Update	PUT /api/students/:id	updateOne() + $set
Delete	DELETE /api/students/:id	deleteOne()

Production systems also need authentication/authorisation, stricter validation, rate limiting, logging, pagination and secure deployment configuration.

15. Write a JavaScript program to sort array elements

Array.prototype.sort() sorts in place. Its default comparison is string-based; numeric arrays should use a comparator.

Ascending and descending numeric order

```
const numbers = [45, 12, 89, 3, 27, 12];
const ascending = [...numbers].sort((a, b) => a - b);
const descending = [...numbers].sort((a, b) => b - a);

console.log("Original:", numbers);
console.log("Ascending:", ascending);
console.log("Descending:", descending);
```

Sample output

```
Original: [45, 12, 89, 3, 27, 12]
Ascending: [3, 12, 12, 27, 45, 89]
Descending: [89, 45, 27, 12, 12, 3]
```

- `a - b` places smaller numbers first; `b - a` places larger numbers first.
- The spread syntax creates a copy, preserving the original array.
- `sort()` mutates the array it is called on. For strings, a comparator such as `a.localeCompare(b)` is useful.

16. Validate a user's registration form before submitting it

Client-side validation checks input before sending it to the server. It improves usability but can be bypassed; the server must repeat validation independently.

registration.html

```
<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8"><meta name="viewport" content="width=device-width, initial-scale=1">
<title>Registration Form</title>
<style>
body {font:16px Arial;max-width:500px;margin:30px auto;padding:0 16px}
label {display:block;margin-top:12px}
input {width:100%;box-sizing:border-box;padding:9px}
#errors {color:#b42318}
</style>
</head>
<body>
<h1>User Registration</h1>
<form id="registerForm" action="/register" method="post" novalidate>
<label for="name">Full name</label>
<input id="name" name="name" autocomplete="name" required>
<label for="email">Email</label>
<input id="email" name="email" type="email" autocomplete="email" required>
<label for="password">Password</label>
<input id="password" name="password" type="password" autocomplete="new-password" required>
<label for="confirmPassword">Confirm password</label>
<input id="confirmPassword" name="confirmPassword" type="password" autocomplete="new-password" required>
<p id="errors" role="alert" aria-live="polite"></p>
<button type="submit">Register</button>
</form>
<script>
const form = document.getElementById("registerForm");
form.addEventListener("submit", function(event) {
  const name = document.getElementById("name").value.trim();
  const email = document.getElementById("email").value.trim();
  const password = document.getElementById("password").value;
  const confirmPassword = document.getElementById("confirmPassword").value;
  const errors = [];

  if (name.length < 2) errors.push("Name must contain at least 2 characters.");
  const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  if (!emailPattern.test(email)) errors.push("Enter a valid email address.");
  if (password.length < 8) errors.push("Password must be at least 8 characters.");
  if (password !== confirmPassword) errors.push("Passwords do not match.");

  document.getElementById("errors").textContent = errors.join(" ");
  if (errors.length > 0) event.preventDefault();
});
</script>
</body></html>
```

Working

- The submit handler reads the entered fields.
- It checks name length, basic email format, password length and matching confirmation.
- If errors exist, the browser displays them and preventDefault() stops submission.
- If all checks pass, the browser submits using POST to /register.
- The server must validate again and store a suitably hashed password, never the plaintext password.

The regular expression is only a basic email-format check; it cannot prove that an address exists.

LAST-MINUTE REVISION

Topic	Remember
Node.js	JavaScript runtime outside browser; V8, event-driven, non-blocking I/O.
Beans	Stateful retains conversational state; stateless does not. JavaBean uses class conventions.
MongoDB	Document-oriented NoSQL database; BSON documents in collections.
DOM	Tree-like document representation that JavaScript can manipulate.
TCP vs UDP	TCP is connection-oriented/reliable; UDP datagrams do not guarantee delivery/order.
npm / REPL	npm init creates package.json; REPL = Read, Evaluate, Print, Loop.
AJAX	Asynchronous request/response; updates part of a page without full reload.
CRUD	Create = insertOne; Read = find; Update = updateOne; Delete = deleteOne.
Numeric sorting	arr.sort((a,b) => a-b) ascending; (b-a) descending.
Form validation	Validate in browser for usability and on server for security.

Answer-writing structure

5 marks: definition + key points + example. 10 marks: introduction + steps/algorithm + complete code + explanation/output + conclusion. Match the detail to the exact wording and your university’s marking scheme.

Technical note: code targets current Node.js conventions and the official MongoDB driver. Ensure MongoDB is running and the .env URI is correct before executing.